saturnCFD v1

MPI and GPU finite-volume CFD with validated methodology

Ananth Narayan

Indian Institute of Technology Madras

31 October 2026

Outline

1. Motivation and scope
2. What is implemented
3. Mesh and configuration
4. Parallel model
5. Finite-volume method
6. Verification and validation
7. Commands and tests
8. Status and roadmap

What saturnCFD is

Goal

A single finite-volume code for scalar transport and incompressible flow, portable across CPU and GPU, parallel through MPI, driven entirely by YAML, and verified before it is trusted.

- Structured hexahedral mesh imported from Gmsh `.msh`.
- 3D from the start; a one-cell-thick mesh is treated as 2D.
- Solvers: Burgers, heat/convection–diffusion, Stokes, incompressible Navier–Stokes.
- Kokkos for CPU/GPU; MPI for distributed memory.
- Every numerical choice is documented, cited and tested.

v0 is preserved on `v0-archive`. v1 is a deliberate rewrite, not a patch on the old AMR container.

Why this order?

1. Burgers: nonlinear convection, shocks, limiters, diffusion.
2. Heat: scalar transport, Péclet effects, boundary layers.
3. Stokes: vector elliptic operator, pressure coupling, linear solver.
4. Incompressible Navier–Stokes: all of the above together.

Idea

Each PDE tests one more ingredient. A failure is localised before the next solver is added.

Compressible flow, AMR and two-phase flow are later milestones.

Implemented substrate

- Gmsh 2.2 reader; structured-hex classification and `checkMesh` report.
- Default `.vtu` + `.json`; optional `-writeVTK` patches and `.pvd`.
- yaml-cpp configuration with a typed schema and generated templates.
- SI-only validation: non-SI input is rejected, not converted.
- Single-dash CLI with help topics and shell completion.
- MPI Cartesian partition and 26-neighbour halo exchange.
- 1/2/4-rank MPI tests, regression tests, Doxygen HTML/PDF.

Status

The mesh/config/comm slice is working. Kokkos fields and the FVM operators are the next implementation step.

Architecture

Layer	Responsibility
`core`	types, SI units, errors, timer
`config`	yaml-cpp, schema, validation, templates
`mesh`	Gmsh, `checkMesh`, structured view, VTK
`comm`	partition, ghost ownership, halo exchange
`fv`	fields, gradient, divergence, Laplacian, fluxes
`linalg`	MPI PETSc systems
`simulation`	solver registry, time loop, diagnostics
`solvers`	Burgers, heat, Stokes, incompressible

A solver owns its equations. It does not own the mesh, the halo, the YAML parser or the linear algebra.

Structured mesh and `checkMesh`

- Gmsh 2.2 ASCII: points, hexahedra, boundary physical groups.
- Classifier walks the cell dual graph and assigns a contiguous (i, j, k).
- Rejects tetrahedra, non-manifold faces, negative volumes and non-contiguous hex meshes.
- Reports counts, patches, volume/area ranges, non-orthogonality, skewness, aspect ratio, structured flag and dimension.

2D rule

A one-cell-thick 3D mesh with `empty` patches is 2D. One internal representation, one solver path.

Gmsh [3]; OpenFOAM's `checkMesh` convention is a methodology reference.

YAML and SI units

```
solver: common
units:
  system: SI
mesh:
  file: channel.msh
time:
  end: 1.0
  cfl: 0.4
```

- yaml-cpp parses; the schema validates.
- Unknown keys are errors.
- `-dumpConfig` generates the template from the same registration the solver reads.
- `-checkConfig` runs all validation before allocation.

SI only in v1

`units.system: SI` or no block is accepted. `CGS`, `length: mm`, etc. are hard errors. A future units layer converts at the configuration boundary, never in a kernel.

MPI decomposition

global grid	$n_x \times n_y \times n_z$
process grid	$p_x \times p_y \times p_z$
local block	$[i_0, i_1] \times [j_0, j_1] \times [k_0, k_1]$

- `MPI_Dims_create` chooses the process grid; `MPI_Cart_create` handles periodicity.
- 2D uses $p_z = 1$; 1D uses $p_y = p_z = 1$.
- Remainders are distributed one cell at a time from the low index.
- No METIS: a structured mesh does not need a graph partition.

Why Cartesian blocks?

Deterministic neighbours, compact messages and a direct cell $\leftrightarrow$ rank mapping.

No shared cells: owner and ghost

- Every global cell has **one owner rank**.
- Only the owner writes the valid value.
- The neighbour keeps a **read-only ghost copy**.
- Two ghost layers support MUSCL/TVD.
- 26-neighbour exchange: faces, edges, corners.

Shared face, not shared cell

Both ranks compute the same face flux from the same valid/ghost pair. Each rank updates only its own cell. No double counting.

Halo order: local periodic copies → packed MPI → physical boundary conditions.

CPU and GPU execution

- Kokkos selects the backend at **build** time: Serial/OpenMP or CUDA/HIP.
- CPU and GPU are separate build directories: `build-cpu` and `build-gpu`.
- `mpirun` launches MPI ranks; the compiled backend decides whether a rank uses a CPU or a GPU.
- Performance model: **one rank per GPU**.
- One GPU: `mpirun -np 1`. Four GPUs: `mpirun -np 4`.
- Halo first implementation: device pack $\rightarrow$ host MPI $\rightarrow$ device unpack.

Case files are hardware-independent

CPU/GPU is not in `case.yaml`. Physics stays portable.

Collocated cell-centred FVM

- Velocity and pressure at cell centroids.
- Face fluxes reconstructed from the two adjacent cells.
- Rhie–Chow correction suppresses checkerboard pressure.
- SIMPLE for steady Stokes/incompressible flow.
- PISO for transient incompressible flow.

Why not MAC?

Staggered MAC gives exact discrete continuity, but it splits the velocity components onto different meshes and fights the vector form, `.msh` import and the 2D/3D rule. Collocated FV matches OpenFOAM and the default Basilisk/Nektar++ approach [11, 1].

Governing equations

$$\text{Burgers:} \quad \frac{\partial u}{\partial t} + \nabla \cdot (u\mathbf{a}) = \nu \nabla^2 u + s$$

$$\text{Heat:} \quad \frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{u} T) = \nabla \cdot (\alpha \nabla T) + S$$

$$\text{Stokes:} \quad -\nabla p + \mu \nabla^2 \mathbf{u} = \mathbf{f}, \qquad \nabla \cdot \mathbf{u} = 0$$

$$\text{Incompressible NS:} \quad \frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u} \otimes \mathbf{u}) = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \frac{\mathbf{f}}{\rho},$$
$$\nabla \cdot \mathbf{u} = 0$$

Projection/pressure correction from Chorin [2]; SIMPLE/PISO from Patankar [7] and Issa [5]; Rhie–Chow [8].

Vector form and linear algebra

Discrete momentum

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \mathbf{C}(\mathbf{u})\mathbf{u} = -\mathbf{G}p + \mathbf{D}\mathbf{u} + \mathbf{f}$$

- One vector unknown per cell: u, v, w.
- One sparsity pattern; block size 3 or three right-hand sides.
- PETSc: `Mat`, `Vec`, `KSP`, `PC`.
- Pressure Poisson: CG + GAMG/HYPRE; Neumann null space handled explicitly.
- No home-grown Krylov solver.

Manufactured solutions: why

Question MMS answers

Does the assembled discrete operator converge to the PDE at the expected order?

- Unit tests check one function; regression checks one accepted result.
- MMS checks the full operator, including faces, ghosts, BCs, geometry and partition boundaries.
- It catches sign/coefficient errors, missed faces and halo bugs that are invisible on one rank.

Salari & Knupp [10]; Roache [9]; Oberkampf & Roy [6].

Manufactured solutions: method

Choose a smooth ϕ, substitute into the PDE, and use the residual as the source:

$$s = \mathcal{L}(\phi).$$

For the periodic Laplacian:

$$\phi = \sin(2\pi x)\sin(2\pi y)\sin(2\pi z), \qquad s = -3(2\pi)^2\phi.$$

Refine h and compute

$$p \approx \frac{\log(e_{\text{coarse}}/e_{\text{fine}})}{\log 2}.$$

Expected orders

Central Laplacian/interpolation: 2. Upwind: 1. TVD on a smooth non-extremum: about 2, with local reduction at extrema.

Validation roadmap

Solver	First validation	Reference
Burgers	viscous traveling wave	exact Cole–Hopf solution
Burgers	inviscid shock	characteristics/conservation
Heat	1D boundary layer	exact solution
Heat	2D/3D MMS	manufactured order
Stokes	Poiseuille/Couette	exact parabolic/linear
Stokes	cavity $Re \to 0$	converged reference
Incompressible	cavity $Re = 100, 1000$	Ghia et al. [4]
Incompressible	Taylor–Green	energy decay/order

Rule

MMS and analytic tests before a physical benchmark. Observations are recorded, including failures and order reduction.

Build and run

```
# CPU build
cmake --preset cpu
cmake --build --preset cpu -j
cmake --install build-cpu \
  --prefix "$HOME/.local"

# GPU build
cmake --preset gpu
cmake --build --preset gpu -j
cmake --install build-gpu \
  --prefix "$HOME/.local"
```

```
# one installed command
mpirun -np 4 saturnCFD case.yaml
mpirun -np 1 saturnCFD case.yaml

# tests
ctest --preset default
ctest --preset regression
ctest --preset mpi
```

The installed binary carries the backend it was compiled with. CPU and GPU are separate build directories; install the one you want to run. Today `cmake --preset default` configures `build-cpu`; the explicit `cpu`/`gpu` presets and the Kokkos backend arrive with the FVM layer.

Help, clean and completion

```
saturnCFD -help
saturnCFD -help build
saturnCFD -help gpu
saturnCFD -help mpi
saturnCFD -help test
saturnCFD -help clean

source <(saturnCFD -completion bash)

cmake --build build-cpu --target clean
rm -rf build-cpu build-gpu
```

No memorisation

Test labels: `ctest --list-presets`. CLI: `saturnCFD -help`. Build: `-help build`. Clean: `-help clean`.

Current test status

Suite	Result
CTest total	16/16 passing
MPI distribution & halo	1, 2, 4 ranks passing
Mesh `checkMesh`	pass/reject fixtures passing
Config SI-only	accept/reject fixtures passing
CLI regression	help, outputs, completion, templates
Doxygen	HTML and PDF build

What is not yet tested

Kokkos fields, FVM gradients/Laplacian, MMS orders, CPU/GPU parity and multi-GPU scaling. These are the next gates.

Status and next steps

Done

- Mesh + `checkMesh`
- YAML schema + SI validation
- CLI/help/completion/install
- MPI partition + 26-neighbour halo
- 1/2/4-rank tests
- Doxygen HTML/PDF

Next

- Kokkos `CellField` + device geometry
- Halo overload for fields
- gradient/divergence/Laplacian
- MMS periodic/Dirichlet/Neumann
- Burgers physics
- PETSc pressure solve

Longer term

Stokes and incompressible NS; multi-GPU cluster scaling; AMR then two-phase flow; compressible flow.

Take-away

What makes this code trustworthy

1. A structured, validated mesh and a strict YAML/SI boundary.
2. An ownership model with no shared cells and a tested halo.
3. A single source for CPU/GPU through Kokkos, not two codes.
4. Verification by manufactured solutions before benchmarks.
5. A CLI and test layout that a user can discover, not memorise.

Questions and discussion

References I

C. D. Cantwell, D. Moxey, A. Comerford, A. Bolis, G. Rocco, G. Mengaldo, D. De Grazia, S. Yakovlev, J.-E. Lombard, D. Ekelschot, B. Jordi, H. Xu, Y. Mohamied, C. Eskilsson, B. Nelson, P. Vos, C. Biotto, R. M. Kirby, and S. J. Sherwin.
Nektar++: An open-source spectral/hp element framework.
Computer Physics Communications, 192:205–219, 2015.

A. J. Chorin.
Numerical solution of the navier–stokes equations.
Mathematics of Computation, 22(104):745–762, 1968.

C. Geuzaine and J.-F. Remacle.
Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities.
International Journal for Numerical Methods in Engineering, 79(11):1309–1331, 2009.

References II

U. Ghia, K. N. Ghia, and C. T. Shin.
High-Re solutions for incompressible flow using the navier–stokes equations and a multigrid method.
Journal of Computational Physics, 48(3):387–411, 1982.

R. I. Issa.
Solution of the implicitly discretised fluid flow equations by operator-splitting.
Journal of Computational Physics, 62(1):40–65, 1986.

W. L. Oberkampf and C. J. Roy.
Verification and Validation in Scientific Computing.
Cambridge University Press, 2010.

S. V. Patankar.
Numerical Heat Transfer and Fluid Flow.
Hemisphere Publishing, 1980.

References III

C. M. Rhie and W. L. Chow.
Numerical study of the turbulent flow past an airfoil with trailing edge separation.
AIAA Journal, 21(11):1525–1532, 1983.

P. J. Roache.
Code verification by the method of manufactured solutions.
Journal of Fluids Engineering, 124(1):4–10, 2002.

K. Salari and P. Knupp.
Code verification by the method of manufactured solutions.
Technical Report SAND2000-1444, Sandia National Laboratories, 2000.

H. G. Weller, G. Tabor, H. Jasak, and C. Fureby.
A tensorial approach to computational continuum mechanics using object-oriented techniques.
Computers in Physics, 12(6):620–631, 1998.